



Namma-Santhe Ledger

Technical Documentation Report

Android • Kotlin

Jetpack Compose

MVVM + Clean Architecture

Offline-First

Firebase Cloud Sync

Gemini OCR

QR Verification

Prepared for Academic & Project Evaluation

May 2026

Table of Contents

- | | |
|--------------------------------|----------------------------------|
| 1. System Overview | 7. App Navigation Flow |
| 2. System Architecture Diagram | 8. OCR Pipeline Architecture |
| 3. UML Class Diagram | 9. QR Confirmation System |
| 4. Use Case Diagram | 10. Security & Tamper Protection |
| 5. Sequence Diagram | 11. Entity-Relationship Diagram |
| 6. Data Flow Diagram | 12. Project Structure |
| | 13. Technology Stack |

1. System Overview

Problem Statement

Rural vendors in Indian markets maintain handwritten ledgers for credit ("udari") tracking. This leads to lost/damaged records, calculation errors, no backups, and delayed payments.

Solution

Namma-Santhe Ledger is an **offline-first Android application** that digitizes credit tracking with automatic balance calculation, customer photo identification, QR-based transaction confirmation, OCR bill scanning, cloud backup, and PDF export.

Key Features

Feature	Description	Technology
Quick Entry	Numeric keypad for <5 sec transaction entry	Jetpack Compose
Customer Management	Photo capture, search, profiles	CameraX, Coil
Credit Tracking	Auto-calculated outstanding balances	Room DB
QR Confirmation	Tamper-proof transaction verification	ZXing, HMAC-SHA256
OCR Bill Scan	Extract text from handwritten/printed bills	Gemini ML + Tesseract
Cloud Sync	Firebase Auth + Firestore backup	Firebase
PDF Export	Shareable ledger reports	iText7
Offline Mode	Full functionality without internet	Room + SQLite

2. System Architecture

The application follows a **4-layer MVVM + Clean Architecture** pattern:

- **Presentation Layer** — Jetpack Compose declarative UI screens
- **ViewModel Layer** — MVVM state management with Kotlin StateFlow
- **Repository Layer** — Data access abstraction (local DB vs cloud)
- **Data Layer** — Room DB, Firebase, Gemini ML, ZXing, iText7

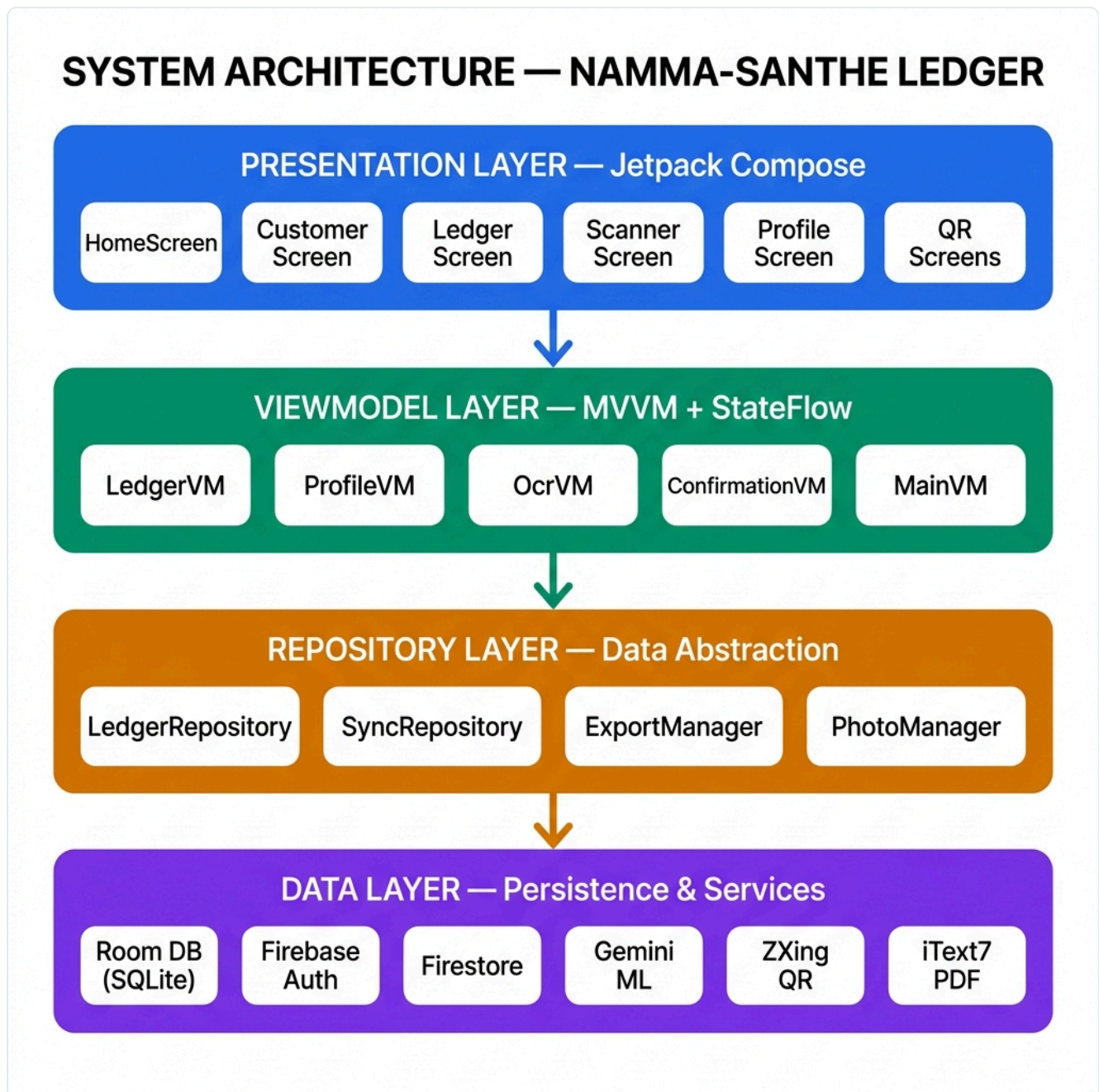


Figure 2.1 — Four-layer system architecture with dependency flow

3. UML Class Diagram

Core entity classes with relationships, QR security classes, and repository pattern.

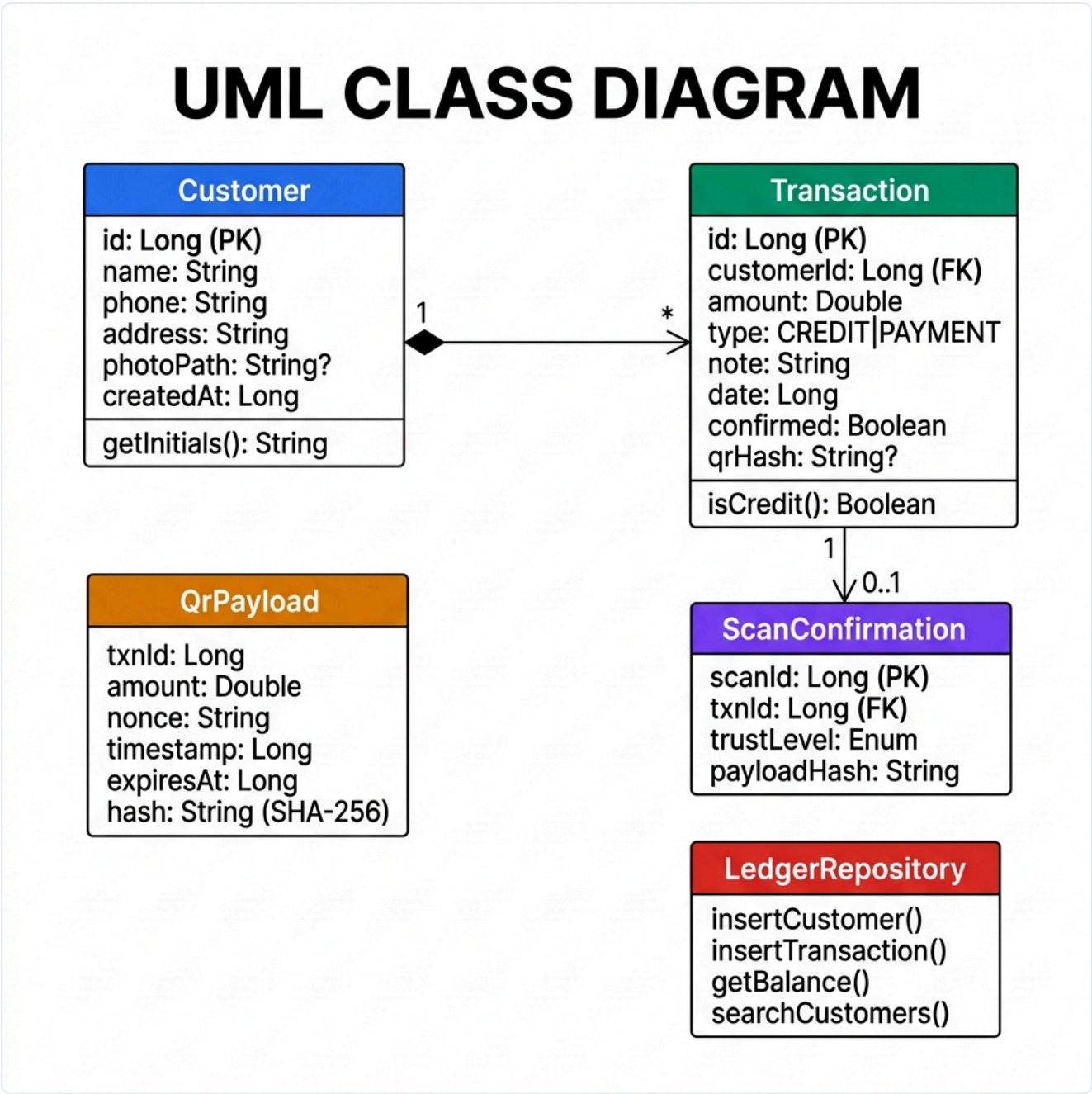


Figure 3.1 — UML Class Diagram showing entity relationships

Key Relationships

From	To	Type	Description
Customer	Transaction	1 → *	One customer has many transactions
Transaction	ScanConfirmation	1 → 0..1	Optional QR confirmation
ViewModel	Repository	Dependency	Data access through abstraction

Repository	Room DAO	Dependency	Database queries via DAOs
------------	----------	------------	---------------------------

4. Use Case Diagram

Two actors: **Vendor** (primary) and **Customer** (secondary).

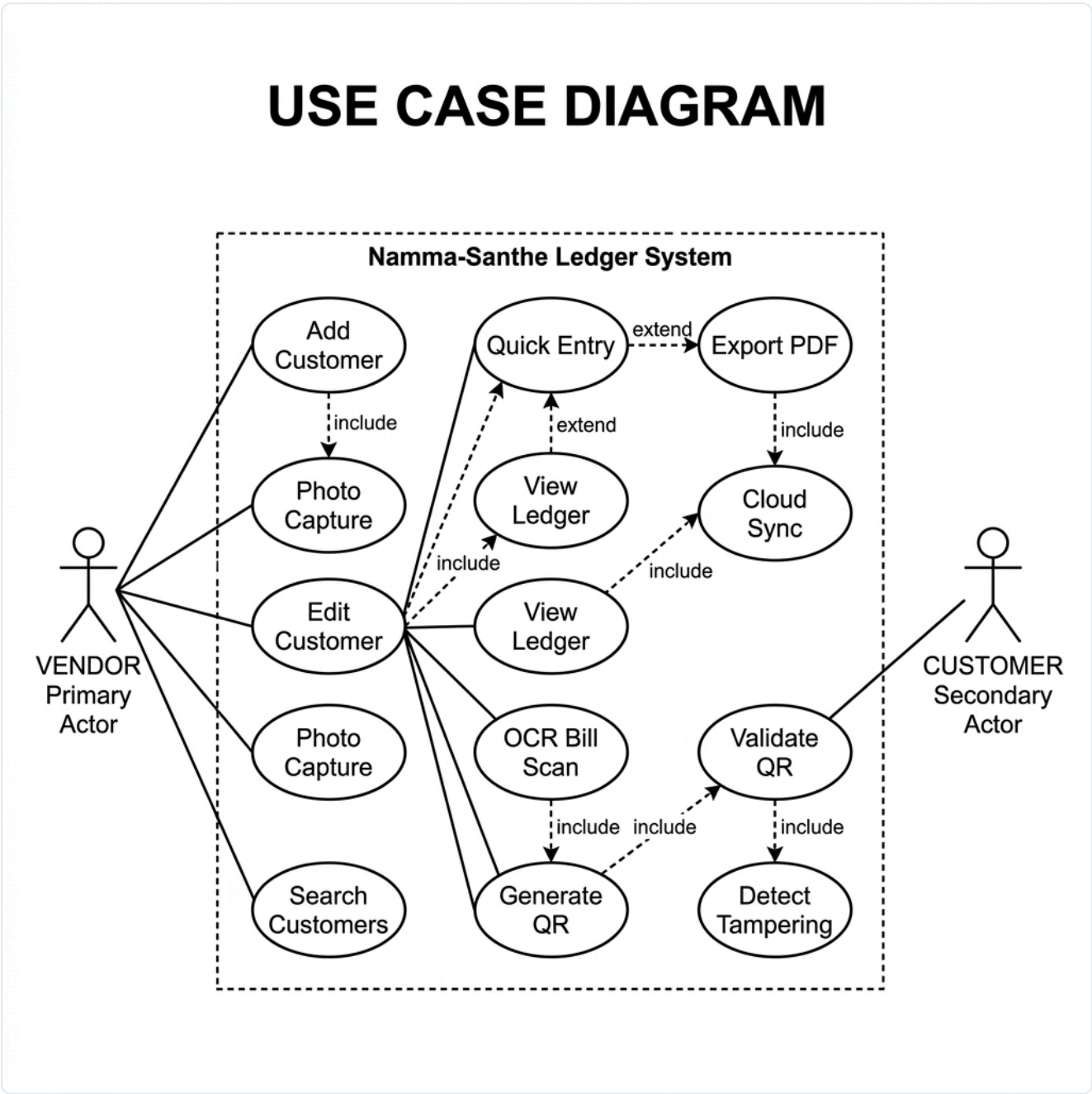


Figure 4.1 — Use Case Diagram with actor-system interactions

Actor Responsibilities

Actor	Use Cases
Vendor	Manage Customers, Process Transactions, Quick Entry, OCR Scan, Generate QR, Export PDF, Cloud Sync
Customer	Scan QR to Confirm Transaction

5. Sequence Diagram — Add Transaction

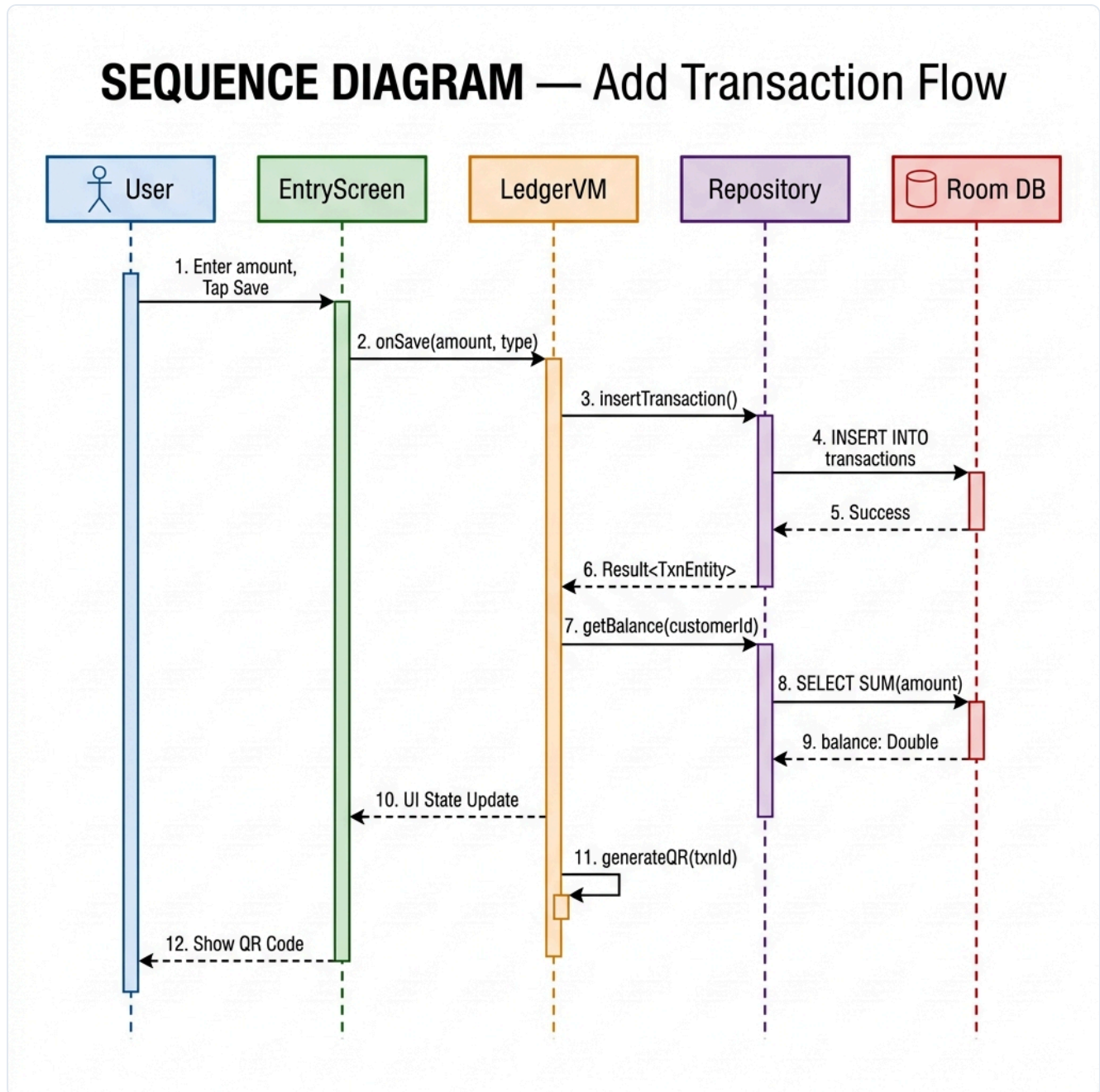


Figure 5.1 — Sequence diagram for Add Transaction flow

Flow Summary

1. User enters amount and taps Save
2. EntryScreen calls `onSave()` on LedgerViewModel
3. LedgerVM inserts transaction via Repository → Room DB
4. Balance recalculated via `SELECT SUM(amount)`
5. UI updates with new balance via `StateFlow`
6. QR code generated with SHA-256 hash

7. QR displayed for customer confirmation (60s TTL)

6. Data Flow Diagram (DFD Level 1)

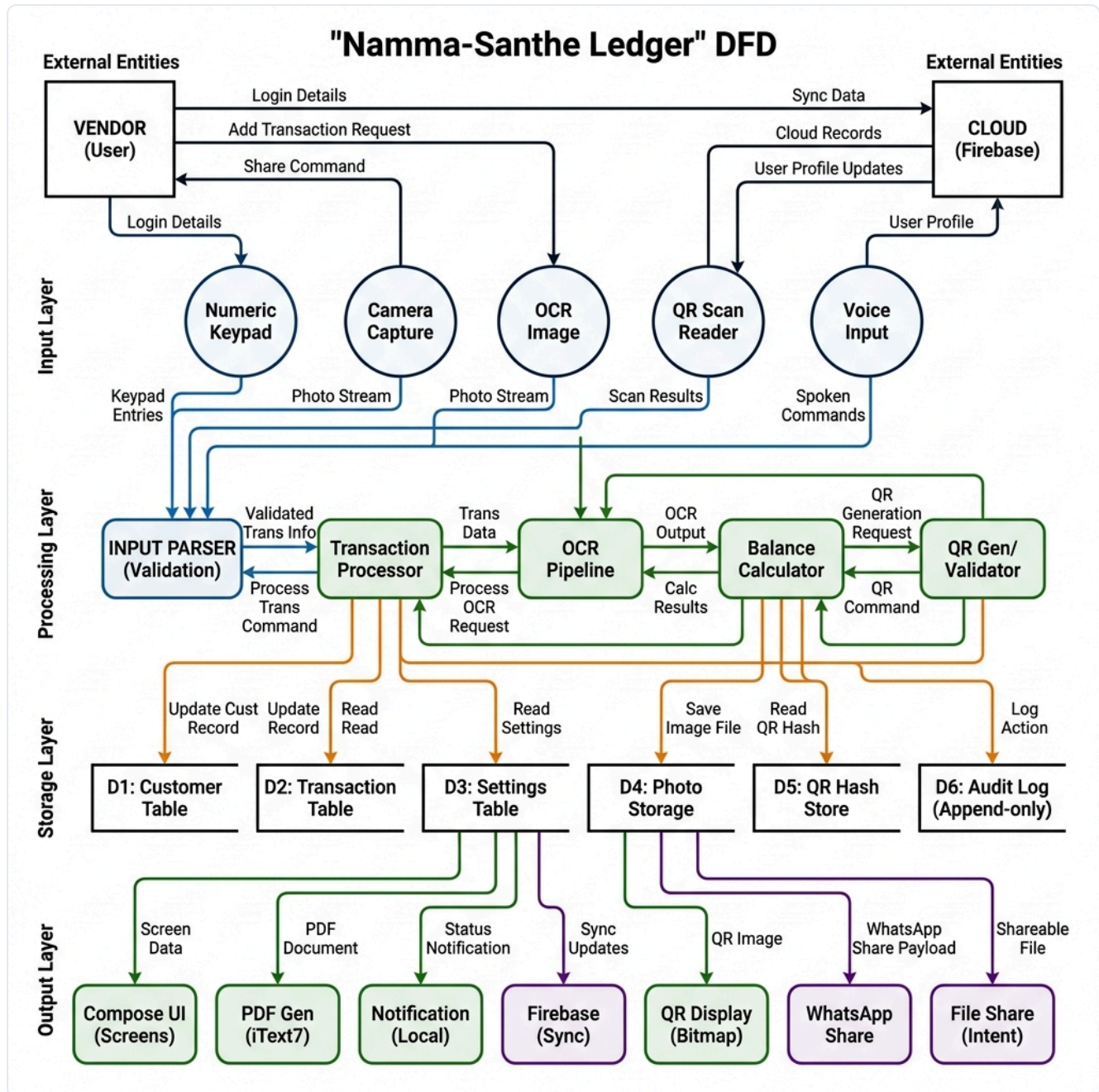


Figure 6.1 — Level 1 Data Flow Diagram

Data Stores

ID	Name	Description
D1	Customer DB	Customer profiles and metadata
D2	Transaction DB	All credit/payment records (append-only)
D3	Photo Storage	Compressed customer/bill images
D4	Settings	App configuration and preferences

D5	QR Hash Store	Used nonces and hash records
D6	Audit Log	Immutable append-only activity log

7. App Navigation Flow

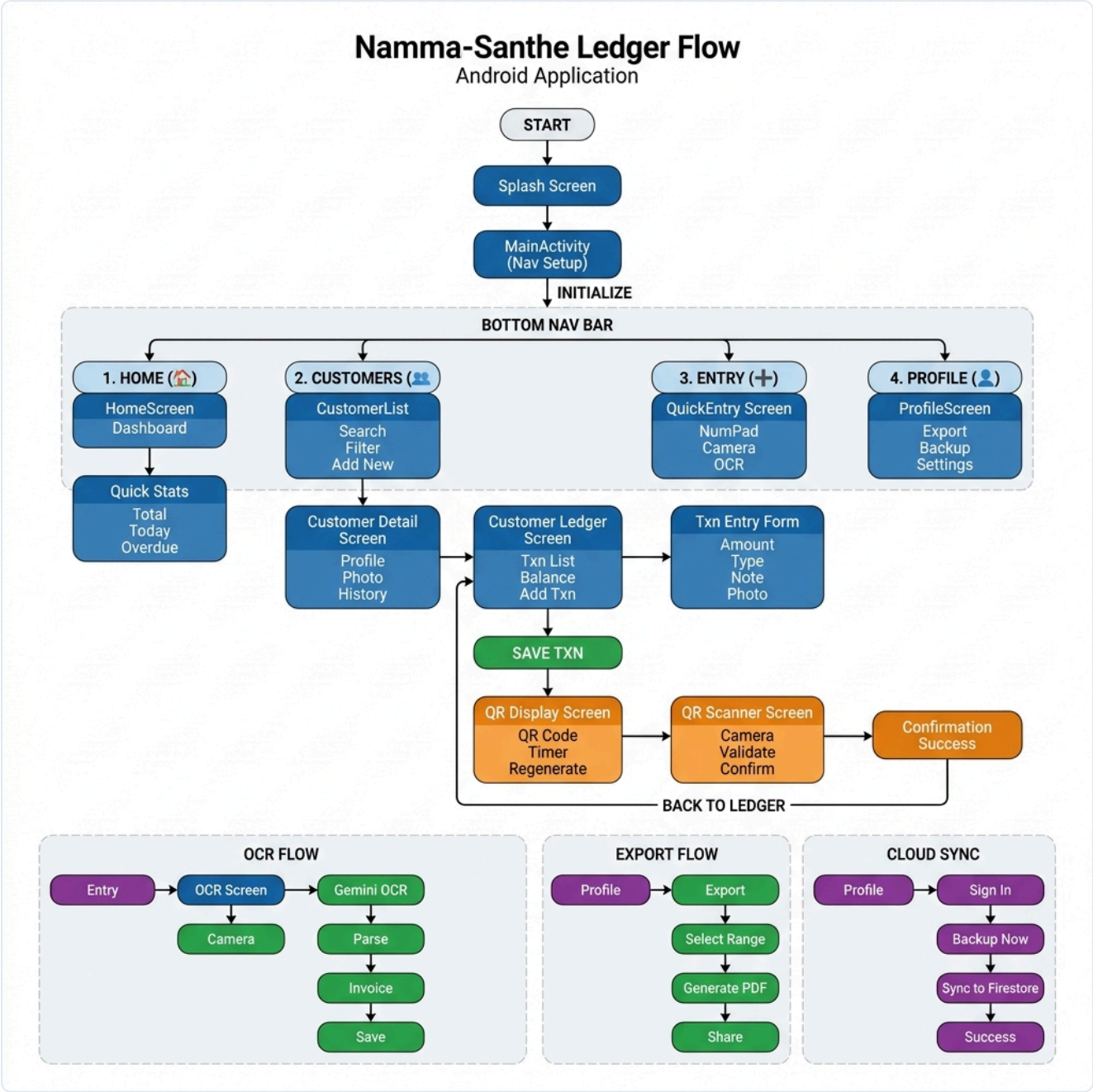


Figure 7.1 — App navigation architecture and screen flow

Navigation Structure

Tab	Screen	Features
Home	Dashboard	Quick stats, total outstanding, today's transactions
Customers	Customer List	Search, filter → Detail → Ledger
Scan	Scanner	OCR bill scanning, camera capture

 Reports	Reports	Analytics, charts, export
--	---------	---------------------------

8. OCR Pipeline Architecture

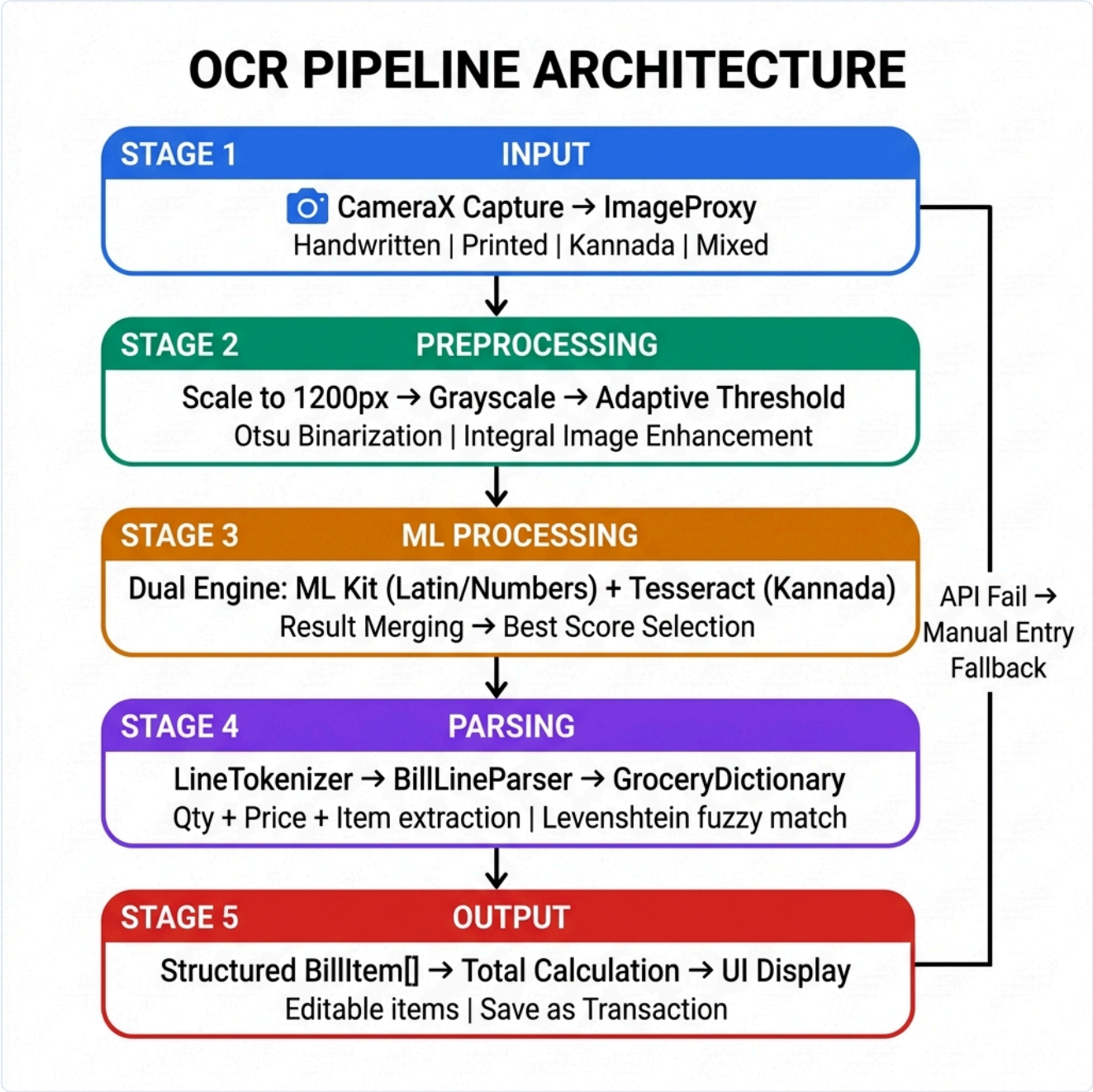


Figure 8.1 — Multi-engine OCR pipeline with preprocessing stages

Pipeline Stages

Stage	Component	Function
1. Input	CameraX	Capture bill image (handwritten, printed, Kannada, mixed)
2. Preprocessing	Image Processor	Scale to 1200px, grayscale, adaptive threshold, Otsu binarization
3. Recognition	Dual Engine	ML Kit (Latin/numbers) + Tesseract (Kannada script) → result merging

4. Parsing	LineTokenizer + BillParser	Quantity, price, item extraction with GroceryDictionary fuzzy match
5. Output	Structured Data	BillItem[] → editable UI → save as transaction

9. QR Confirmation System

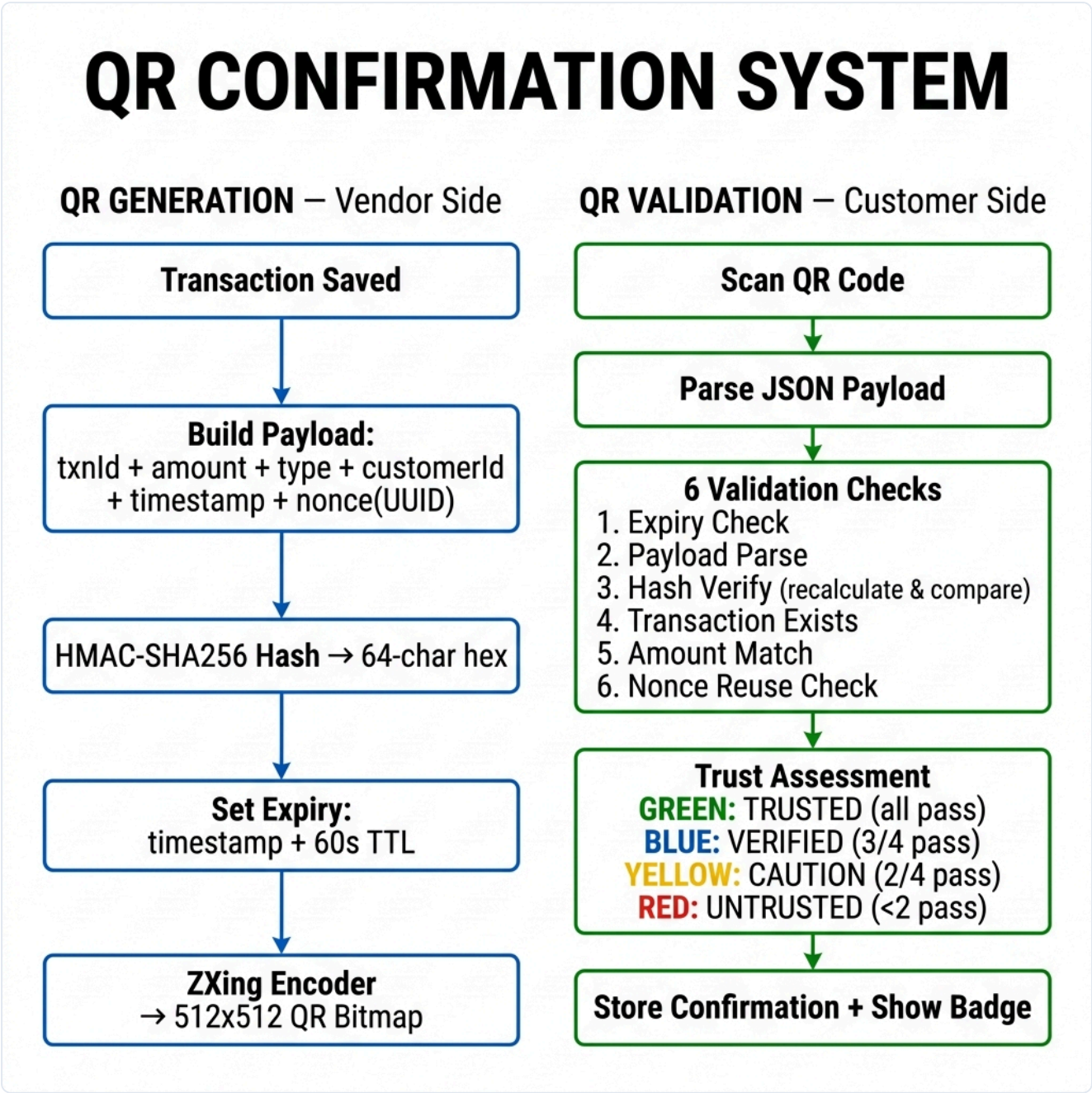


Figure 9.1 — QR generation and validation architecture

Validation Checks

#	Check	Reject Condition
1	Expiry	currentTime > expiresAt (60s TTL)
2	Payload Parse	Malformed JSON payload
3	Hash Verify	SHA-256 recalculation ≠ stored hash
4	Txn Exists	txnId not found in local database

5	Amount Match	QR amount $\neq$ DB stored amount
6	Reuse Check	Nonce already consumed

Trust Assessment Levels

Level	Condition	Badge
TRUSTED	All checks pass	 Green
VERIFIED	3/4 checks pass	 Blue
CAUTION	2/4 checks pass	 Yellow
UNTRUSTED	<2 checks pass	 Red

10. Security & Tamper Protection

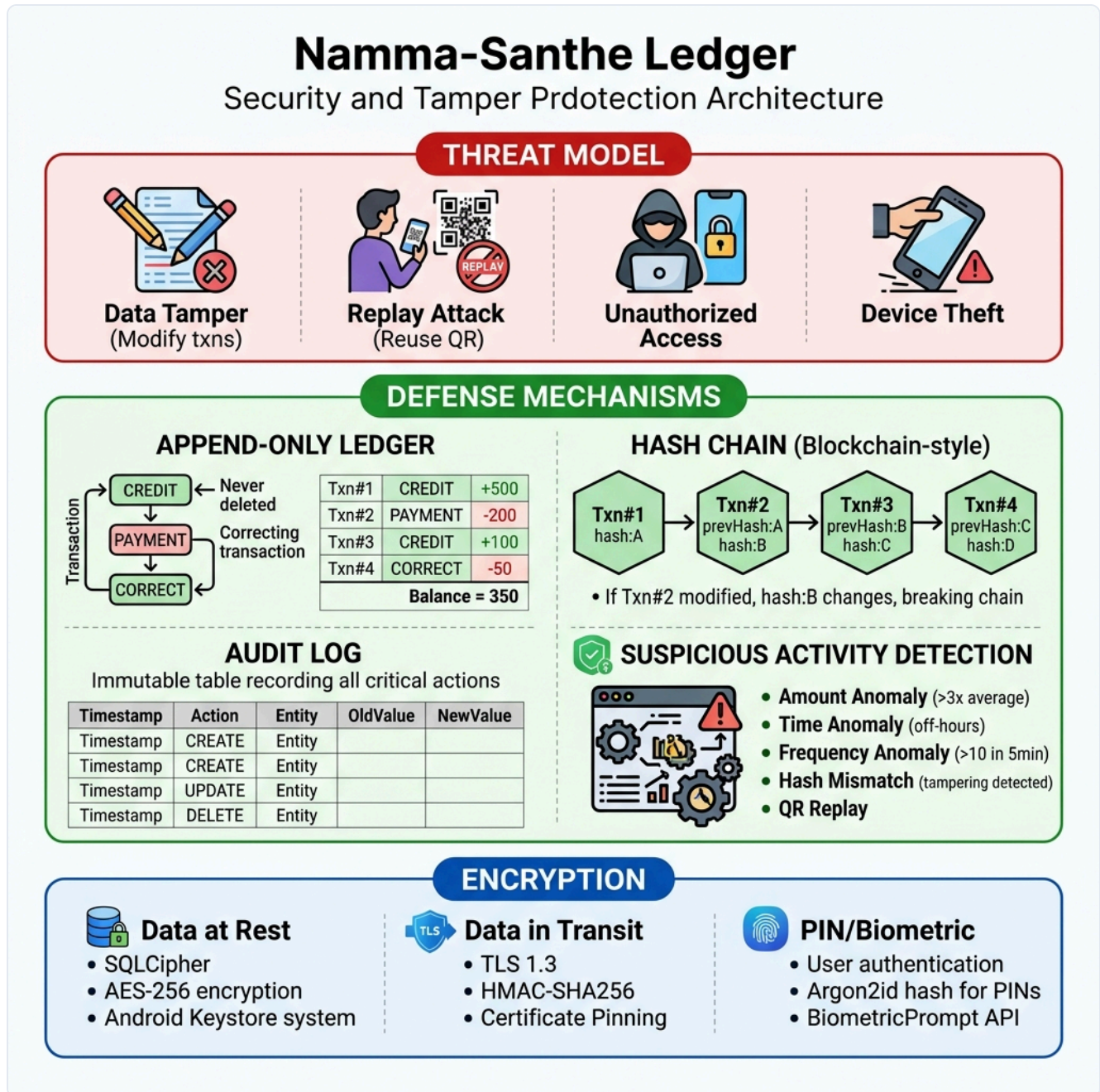


Figure 10.1 — Security architecture and defense mechanisms

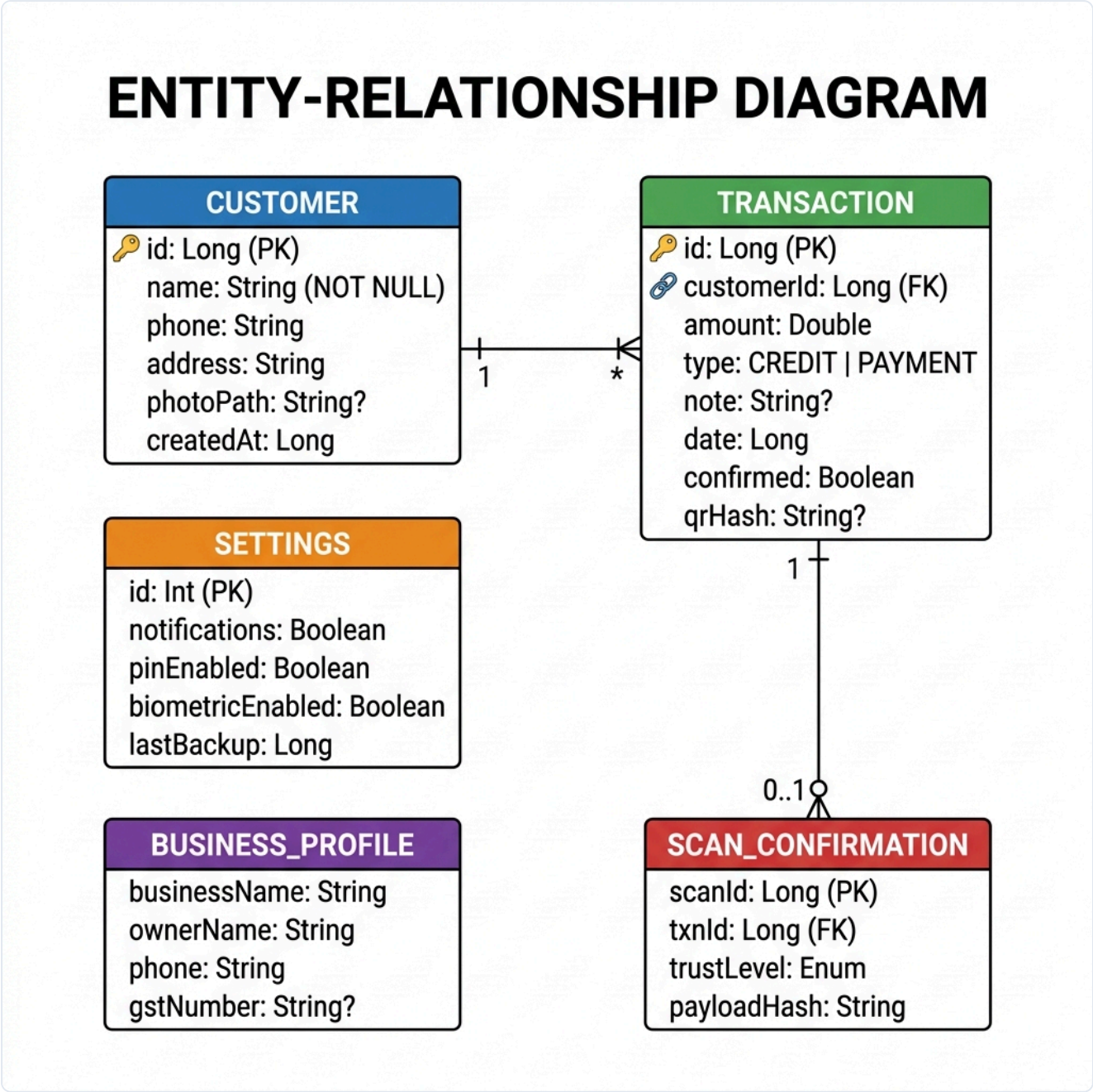
Defense Mechanisms

Mechanism	Description
Append-Only Ledger	Transactions never deleted; corrections create new entries
Hash Chain	Blockchain-style: each txn hash includes previous hash. Modification breaks chain.
Audit Log	Immutable log of all CREATE/UPDATE/DELETE with old/new values
Anomaly Detection	Amount >3× avg, off-hours, >10 txns/5min, hash mismatch, QR replay

Encryption

Layer	Technology
Data at Rest	SQLCipher AES-256, Android Keystore (hardware-backed)
Data in Transit	TLS 1.3, HMAC-SHA256 signed payloads, Certificate Pinning
Authentication	Argon2id memory-hard PIN hash, BiometricPrompt API

11. Entity-Relationship Diagram



ScanConfirmation	scanId (PK), txnId (FK), trustLevel, payloadHash	Belongs to Transaction
------------------	--	------------------------

12. Project Structure

```
app/src/main/java/com/nammasanthe/ledger/
├── MainActivity.kt           # Entry point, Navigation host
├── NammaSantheApp.kt        # Application class
├── data/
│   ├── database/AppDatabase.kt # Room database definition
│   ├── entity/               # Customer, TxnEntity, Confirmation
│   └── dao/                  # CustomerDao, TransactionDao
├── repository/
│   ├── LedgerRepository.kt    # Main data operations (CRUD)
│   └── DataExportManager.kt   # PDF export + file sharing
├── viewmodel/
│   ├── LedgerViewModel.kt     # Ledger screen business logic
│   ├── ProfileViewModel.kt    # Profile, auth, settings
│   ├── ConfirmationViewModel.kt # QR generation and scanning
│   └── OcrViewModel.kt        # OCR processing pipeline
├── ui/
│   ├── screens/              # HomeScreen, CustomerScreen, etc.
│   ├── components/           # Reusable Compose components
│   ├── nav/Routes.kt         # Navigation route constants
│   └── theme/                 # Colors, Typography, Theme
├── sync/
│   ├── FirebaseAuthManager.kt # Sign up, Sign in, Auth state
│   └── FirebaseSyncManager.kt # Cloud sync (Firestore)
├── security/
│   ├── QrGenerator.kt         # QR code generation + signing
│   ├── QrValidator.kt         # QR validation + trust assessment
│   └── QrScannerAnalyzer.kt    # CameraX QR scanning
├── ocr/
│   ├── OcrPipeline.kt         # Orchestrates ML Kit + Tesseract
│   ├── MLKitOcr.kt            # Google ML Kit text recognition
│   ├── TesseractOcr.kt       # Tesseract for Kannada script
│   ├── BillParser.kt          # Multi-line bill parsing
│   ├── BillLineParser.kt      # Single-line item extraction
│   ├── LineTokenizer.kt       # Token-based line analysis
│   └── GroceryDictionary.kt    # 150+ item multilingual dictionary
├── gemini/
│   ├── GeminiService.kt        # Gemini API for advanced OCR
│   └── GeminiSettingsStore.kt   # API key + settings storage
└── util/
    ├── PhotoManager.kt        # Photo capture + compression
    └── PdfGenerator.kt         # iText7 PDF generation
```

13. Technology Stack

Layer	Technology	Purpose
-------	------------	---------

UI Framework	Jetpack Compose	Modern declarative Android UI
State Management	Kotlin StateFlow	Reactive UI state updates
Architecture	MVVM + Repository	Clean separation of concerns
Local Database	Room (SQLite)	Offline-first data persistence
OCR Engine	Tesseract + ML Kit	Dual-engine text recognition
AI/ML	Gemini 2.5 Flash	Advanced OCR + translation
QR Codes	ZXing	QR generation and scanning
PDF Export	iText7	Invoice/report generation
Auth	Firebase Auth	Email/password authentication
Cloud Sync	Firestore	Real-time cloud database
Camera	CameraX	Photo capture + QR scanning
Image Loading	Coil	Async image loading + caching